

# Neural Networks from Scratch

Quelques éléments de théorie

21.02.2026

# Contents



|                                                                                            |    |
|--------------------------------------------------------------------------------------------|----|
| 0.a Objectifs .....                                                                        | 5  |
| 1 Réseau de neurones .....                                                                 | 6  |
| 1.a Qu'est ce qu'un réseau de neurones ? .....                                             | 7  |
| 1.b Comment entraine t'on un réseau de neurones ? .....                                    | 9  |
| 1.c Dérivée .....                                                                          | 11 |
| 2 Autodiff - calcul automatique de la dérivée .....                                        | 13 |
| 2.a Formules usuelles pour la dérivée de fonctions de $\mathbb{R}$ dans $\mathbb{R}$ ..... | 14 |
| 2.b Représentation interne d'un graphe d'opérations .....                                  | 15 |
| 2.c .....                                                                                  | 17 |
| 2.d Dérivée .....                                                                          | 18 |
| 2.e Exercice 01.a : Implémentation en Python .....                                         | 19 |
| 2.f Exercice 01.b : Test de l'autodiff .....                                               | 20 |
| 2.g Exercice 01.c : estimation d'un paramètre .....                                        | 21 |
| 2.h Quelles sont les améliorations possibles dans le code ? .....                          | 22 |
| 3 Dérivée de fonctions de plusieurs variables .....                                        | 23 |
| 3.a Derivee multi-variables .....                                                          | 24 |

## Contents (ii)



|     |                                                                    |    |
|-----|--------------------------------------------------------------------|----|
| 3.b | Exercice 02.a : Implémentation de la dérivée multi-variables ..... | 25 |
| 3.c | Exercice 02.b : optimisation .....                                 | 27 |
| 3.d | Gradient et Jacobienne .....                                       | 28 |
| 3.e | Intérêt du gradient et de la jacobienne .....                      | 30 |
| 4   | Dérivée directionnelle .....                                       | 31 |
| 4.a | Dérivée directionnelle .....                                       | 32 |
| 4.b | Formules usuelles pour la dérivée directionnelle .....             | 34 |
| 4.c | Intérêt de la dérivée directionnelle .....                         | 35 |
| 4.d | Sans dérivée directionnelle : .....                                | 37 |
| 4.e | Avec la dérivée directionnelle .....                               | 38 |
| 4.f | Exercice 03.a dérivée par rapport à un tenseur .....               | 39 |
| 4.g | Exercice 03.b : Descente de gradient avec des tenseurs .....       | 41 |
| 5   | Backpropagation .....                                              | 42 |
| 5.a | Constat sur la répétition du même calcul .....                     | 43 |
| 5.b | Dérivée du réseau de neurones .....                                | 44 |
| 5.c | Qu'est ce que la backpropagation ? .....                           | 46 |

# Contents (iii)



|                                                         |    |
|---------------------------------------------------------|----|
| 5.d Apport d'une librairie de réseaux de neurones ..... | 48 |
| 6 Fin de la partie théorique .....                      | 49 |
| 6.a Sources .....                                       | 50 |

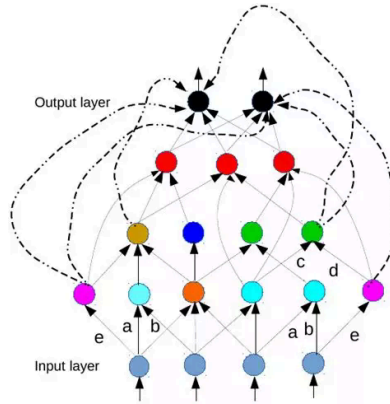
# Objectifs



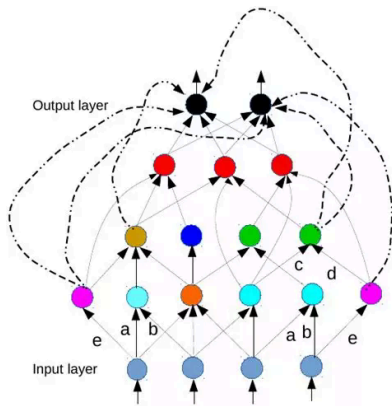
- S'amuser
- Uniquement les aspects théoriques des réseaux de neurones (uniquement Python, Numpy)
- Implémenter un autodiff
- Incrémental
- Comprendre l'apport d'une librairie de réseaux de neurones

# **1 Réseau de neurones**

# Qu'est ce qu'un réseau de neurones ?



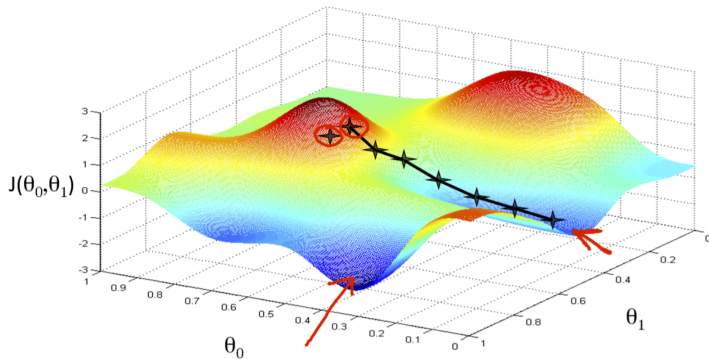
## Qu'est ce qu'un réseau de neurones ? (ii)



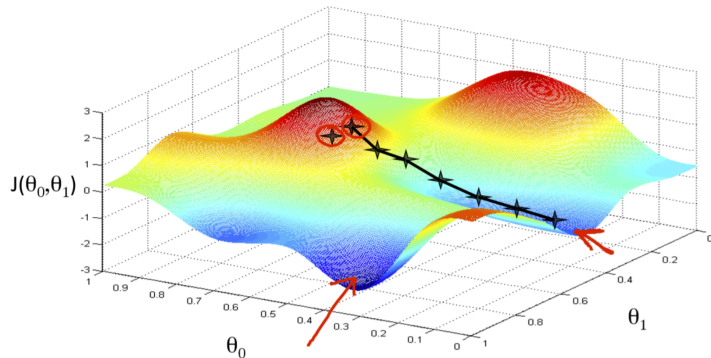
- Un réseau de neurones est une fonction paramétrée  $y(w, x)$
- Il est en général composé de plusieurs couches empilées, correspondant chacune à une fonction paramétrée  $f_i(w_i, \cdot)$
- On peut le représenter sous forme d'un DAG (graphe orienté acyclique)



# Comment entraine t'on un réseau de neurones ?



# Comment entraîne t'on un réseau de neurones ? (ii)



- On définit une fonction de coût  $\mathcal{L}(w)$
- On minimise cette fonction de coût par rapport aux paramètres  $w$  du réseau



## Question

A quoi sert la dérivée d'une fonction pour l'entraînement d'un réseau de neurones ?

## Dérivée (ii)



### Question

A quoi sert la dérivée d'une fonction pour l'entraînement d'un réseau de neurones ?

- Donner la pente au voisinage d'un point

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

- Donner une approximation locale de la fonction

$$f(x+h) \approx f(x) + f'(x) \cdot h$$

- Permettre de minimiser une fonction de coût

$$x_{n+1} = x_n - \eta \cdot f'(x_n)$$

- Dans le réseau de neurones, la variable que l'on modifie est  $w$  (les paramètres du réseau) et non pas  $x$ .

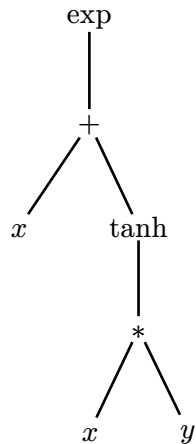
## **2 Autodiff - calcul automatique de la dérivée**

# Formules usuelles pour la dérivée de fonctions de $\mathbb{R}$ dans $\mathbb{R}$

| Nom de la règle | Formule                          | Dérivée                    |
|-----------------|----------------------------------|----------------------------|
| Constante       | $C$                              | $0$                        |
| Carré           | $x^2$                            | $2x$                       |
| Somme           | $f(x) + g(x)$                    | $f'(x) + g'(x)$            |
| Produit         | $f(x)g(x)$                       | $f'(x)g(x) + f(x)g'(x)$    |
| Composition     | $f(g(x))$                        | $f'(g(x)).g'(x)$           |
| Sigmoïde        | $\sigma(x) = \frac{1}{1+e^{-x}}$ | $\sigma(x)(1 - \sigma(x))$ |

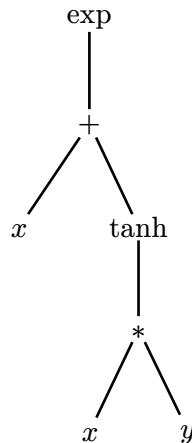
# Représentation interne d'un graphe d'opérations

$$\exp(x + \tanh(xy))$$



## Représentation interne d'un graphe d'opérations (ii)

$\exp(x + \tanh(xy))$



### Question

Comment représenter cette expression en Python ?



On pourrait représenter l'addition ainsi, où `Op` implémente la dérivée `diff()`

```
class Add(Op):  
    def __init__(self, left: Op, right: Op):  
        self.left = left  
        self.right = right  
  
    def diff(self):  
        return Add(self.left.diff(), self.right.diff())
```

### Question

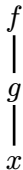
Comment représenter la fonction exponentielle ? Et la variable  $x$  / fonction  $x \mapsto x$  ?

# Dérivée

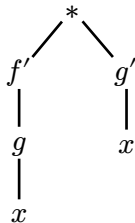


Pour dériver, on transforme récursivement un graphe en un autre

$$f(g(x))$$



$$f'(g(x))g'(x)$$



## Exercice 01.a : Implémentation en Python



Sur le modèle ci-dessous, implémentez :

- la dérivée d'une multiplication
- la dérivée de  $x \rightarrow x^2$

```
class Add(Op):
    def __init__(self, left: Op, right: Op):
        self.left = left
        self.right = right

    def __repr__(self):
        return f"({self.left} + {self.right})"

    def diff(self):
        return Add(self.left.diff(), self.right.diff())

    def eval(self):
        return self.left.eval() + self.right.eval()
```

## Exercice 01.b : Test de l'autodiff



- Avec votre code Python, calculez la dérivée de l'expression suivante :  $(x + 2) * (x + 3)$
- Vérifiez qu'elle correspond bien à

$$(x + 3) + (x + 2) = 2x + 5$$

- Quid de l'expression  $(x + 1) * (y + 2)$  ?

## Exercice 01.c : estimation d'un paramètre



Estimer la moyenne  $\mu$  d'un jeu de données  $X_1, \dots, X_N$  où

- les  $X_i$  sont iid
- et sont tirées selon  $\mathcal{N}(\mu, 1)$

**Objectif** : minimiser une fonction de coût  $\mathcal{L}$

$$\mathcal{L}(\hat{\mu}) = \sum_i (X_i - \hat{\mu})^2$$

Vérifier que  $\hat{\mu} \approx \mu$

# Quelles sont les améliorations possibles dans le code ?



- Enlever automatiquement les 0 des additions
- Simplifier automatiquement les multiplications par 1
- Surcharger les opérateurs Python (+, \*, ...)
- Afficher le graphe de calcul (le fameux DAG)
- Permettre de dériver par rapport à plusieurs variables

### **3 Dérivée de fonctions de plusieurs variables**



Pour calculer une approximation locale de la fonction  $x, y \mapsto f(x, y)$

$$f(x + \varepsilon_x, y + \varepsilon_y) \approx f(x, y) + \frac{\partial f}{\partial x} \varepsilon_x + \frac{\partial f}{\partial y} \varepsilon_y$$

Similaire à

$$f(x + \varepsilon) \approx f(x) + f'(x) \cdot \varepsilon$$



## Exercice 02.a : Implémentation de la dérivée multi-variables

### Question

Comment modifier le code existant pour implémenter la dérivée multi-variables ?

## Exercice 02.a : Implémentation de la dérivée multi-variables (ii)

### Question

Comment modifier le code existant pour implémenter la dérivée multi-variables ?

Une solution, dans la classe `Variable` :

```
def diff(self) -> Variable:  
    return Variable(name="one", value=1.0)
```

```
def diff(self, wrt: Variable) -> Op:  
    if self == wrt:  
        return Variable(name="one", value=1.0)  
    else:  
        return Variable(name="zero", value=0.0)
```

Tester sur une fonction de deux variables, par exemple :

$$f(x, y) = (x + 2)(2y + 3)$$

Résultats attendu :  $\frac{\partial f}{\partial x} = 2y + 3$  et  $\frac{\partial f}{\partial y} = 2(x + 2)$ .

## Exercice 02.b : optimisation



On souhaite ajuster une droite  $y = ax + b$  sur un jeu de données.

**Objectif** : Etant les données générées (voir exercice), minimiser une fonction de coût  $\mathcal{L}$

$$\mathcal{L}(a, b) = \sum_i (y_i - (ax_i + b))^2$$



On souhaite calculer une approximation locale de la fonction  $x \rightarrow f(x)$  où  $x$  est un vecteur et contient de nombreuses variables.

Lorsque  $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ ,  $Jf(x)$  est une matrice

$$f(x + \varepsilon) \approx f(x) + Jf(x) \cdot \varepsilon$$

Lorsque  $f : \mathbb{R}^n \rightarrow \mathbb{R}$ ,  $\nabla f(x)$  est un vecteur de  $\mathbb{R}^n$

$$f(x + \varepsilon) \approx f(x) + \nabla f(x) \cdot \varepsilon$$

## Gradient et Jacobienne (ii)



On souhaite calculer une approximation locale de la fonction  $x \rightarrow f(x)$  où  $x$  est un vecteur et contient de nombreuses variables.

Lorsque  $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ ,  $Jf(x)$  est une matrice

$$f(x + \varepsilon) \approx f(x) + Jf(x).\varepsilon$$

Lorsque  $f : \mathbb{R}^n \rightarrow \mathbb{R}$ ,  $\nabla f(x)$  est un vecteur de  $\mathbb{R}^n$

$$f(x + \varepsilon) \approx f(x) + \nabla f(x).\varepsilon$$

Similaire à lorsque  $f : \mathbb{R} \rightarrow \mathbb{R}$

$$f(x + \varepsilon) \approx f(x) + f'(x).\varepsilon$$

# Intérêt du gradient et de la jacobienne



## Question

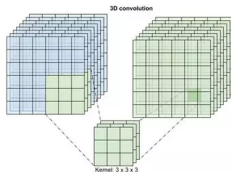
Pourquoi utiliser le gradient/jacobienne ?

- Permet de manipuler des fonctions à plusieurs variables à un autre niveau d'abstraction

Inconvénient dans ce cadre :

- il faut manipuler des matrices pour la Jacobienne
- des vecteurs pour le gradient

Alors que l'on a des plutôt des tenseurs en Machine Learning



## 4 Dérivée directionnelle

# Dérivée directionnelle



$f : \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}^k$  est une **fonction de deux vecteurs** :

$$x, y \mapsto f(x, y)$$

On définit sa dérivée directionnelle par rapport à  $x$  ainsi :

$$\frac{\partial f}{\partial x}(x, y).v = \lim_{h \rightarrow 0} \frac{f(x + hv) - f(x)}{h}$$



## Dérivée directionnelle (ii)



$f : \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}^k$  est une **fonction de deux vecteurs** :

$$x, y \mapsto f(x, y)$$

On définit sa dérivée directionnelle par rapport à  $x$  ainsi :

$$\frac{\partial f}{\partial x}(x, y).v = \lim_{h \rightarrow 0} \frac{f(x + hv) - f(x)}{h}$$

On utilisera aussi la notation plus légère

$$\frac{\partial f}{\partial x}(x, y).\varepsilon = \partial_x f(x, y).\varepsilon$$

# Formules usuelles pour la dérivée directionnelle

$$f(x + \varepsilon) \approx f(x) + \partial f(x) \cdot \varepsilon$$

| Nom de la règle        | Formule       | Dérivée                                                             |
|------------------------|---------------|---------------------------------------------------------------------|
| Constante              | $C$           | 0                                                                   |
| Application linéaire   | $A.x$         | $A.\varepsilon$                                                     |
| Application bilinéaire | $B(x_1, x_2)$ | $B(x_1, \varepsilon_2) + B(\varepsilon_1, x_2)$                     |
| Norme au carré         | $\ x\ ^2$     | $2x.\varepsilon$                                                    |
| Somme                  | $f(x) + g(x)$ | $\partial f(x)\varepsilon + \partial g(x)\varepsilon$               |
| Produit (matriciel)    | $f(x).g(x)$   | $(\partial f(x).\varepsilon).g(x) + f(x).\partial g(x).\varepsilon$ |
| Composition            | $f(g(x))$     | $\partial f(g(x)).\partial g(x).\varepsilon$                        |

\*La non-commutativité doit être respectée pour les produits.

# Intérêt de la dérivée directionnelle



## Question

Quels sont les avantages de la dérivée directionnelle ?

## Intérêt de la dérivée directionnelle (ii)



### Question

Quels sont les avantages de la dérivée directionnelle ?

- Rester à un niveau d'abstraction tenseurs/matrices/vecteurs sans se noyer dans les indices
- Manier des tenseurs directement, sans avoir à reshape en vecteur

⇒ Gain en lisibilité et en clarté.

## Sans dérivée directionnelle :



$$y(k) = \sigma \left( \sum_i w_{k,i} \sigma \left( \sum_j w_{i,j}^0 x_j + b_i^0 \right) + b_k^1 \right)$$

La dérivée par rapport à  $w_{p,q}^0$  est :

$$\frac{\partial y(k)}{\partial w_{m,n}^0} = \sigma' \left( \sum_i w_{k,i} \sigma \left( \sum_j w_{i,j}^0 x_j + b_i^0 \right) + b_k^1 \right) \cdot w_{k,m}^1 \cdot \sigma' \left( \sum_j w_{m,j}^0 x_j + b_m^0 \right) \cdot x_n$$

# Avec la dérivée directionnelle



Formule sans indices :

$$y = \sigma(w_1 \sigma(w_0 x + b_0) + b_1)$$

Avec la dérivée directionnelle, la dérivée par rapport à  $w^0$  s'écrit :

$$\frac{\partial y}{\partial w_0} = \sigma'(w_1 \sigma(w_0 x + b_0) + b_1) \cdot w_1 \cdot \sigma'(w_0 x + b_0) x$$

## Exercice 03.a dérivée par rapport à un tenseur



### Question

Que faut t'il modifier dans le code pour dériver par rapport des à des tenseurs/matrices/vecteurs ?

Rappel :

$$f(x + \varepsilon) \approx f(x) + \partial f(x) . \varepsilon$$

## Exercice 03.a dérivée par rapport à un tenseur (ii)

### Question

Que faut t'il modifier dans le code pour dériver par rapport des à des tenseurs/matrices/vecteurs ?

Rappel :

$$f(x + \varepsilon) \approx f(x) + \partial f(x). \varepsilon$$

```
def diff(self, wrt: Variable) -> Op:
    if self == wrt:
        return Variable(name="one", value=1.0)
    else:
        return Variable(name="zero", value=0.0)
```

```
51
52+ def diff(self, wrt: Variable, direction: Variable) -> Op:
53     if self == wrt:
54+         return direction
55     else:
56+         return ConstantZero()
```

NB : Lorsque l'on dérivait par rapport à une variable réelle, on avait implicitement la direction  $\varepsilon = 1$ . Dans le cas tensoriel, il faut prendre en compte que c'est un vecteur.

$$f(x + h\varepsilon) \approx f(x) + h\partial f(x). \varepsilon$$



## Exercice 03.b : Descente de gradient avec des tenseurs

Même exercice que Exercice 02b, mais en maniant des tenseurs plutôt que des variables individuelles.

**Algorithme** : estime `a_true`, `b_true`, `c_true`, qui sont néanmoins des variables réelles et non pas des vecteurs/matrice/tenseurs.

**Objectif de l'exercice** :

- Observer la différence de complexité de l'expression de la dérivée en comparaison de ce que l'on avait dans l'exercice 02b.
- Décommenter le `break` dans la boucle après avoir enlevé les `print` pour laisser l'algo tourner plus longtemps.

## 5 Backpropagation

# Constat sur la répétition du même calcul



Réseau de neurones : empilement de couches

$$y(w, x) = f_n(\dots f_3(w_3, f_2(w_2, f_1(w_1, f_0(w_0, x)))) \dots)$$

Ou en définissant le réseau de manière récursive

$$y(w, x) = y_n(w_{0:n}, x) = f_n(w_n, y_{n-1}(w_{0:n-1}, x))$$

# Dérivée du réseau de neurones



On dérive par rapport aux paramètres d'une couche  $k < n$  :

$$\frac{\partial y_n}{\partial w_k} = \frac{\partial f_n}{\partial y} \cdot \frac{\partial y_{n-1}}{\partial w_k}$$

## Question

Que remarque t'on ?

## Dérivée du réseau de neurones (ii)



On dérive par rapport aux paramètres d'une couche  $k < n$  :

$$\frac{\partial y_n}{\partial w_k} = \frac{\partial f_n}{\partial y} \cdot \frac{\partial y_{n-1}}{\partial w_k}$$

### Question

Que remarque t'on ?

Une formule récursive apparaît

$$\frac{\partial y_n}{\partial w_k} = \left( \prod_{i=k+1}^n \frac{\partial f_i}{\partial y_i} \right) \frac{\partial f_k}{\partial w_k}$$

⇒ On aimerait mémoriser (*memoize*) le résultat pour le réutiliser.

⇒ La dérivée de la fonction et la fonction gagneraient à se ressembler.

# Qu'est ce que la backpropagation ?



## Question

Qu'est ce que la backpropagation ?

## Qu'est ce que la backpropagation ? (ii)



### Question

Qu'est ce que la backpropagation ?

C'est un algorithme permettant de calculer efficacement la dérivée d'un réseau de neurones :

- la structure par couches permet de dériver une seule couche à la fois (et multiplier par les dérivées des couches suivantes déjà calculées)
- en mémorisant les résultats intermédiaires dans le produit :

$$\left( \frac{\partial y_n}{\partial w_k} \right) = \left( \prod_{i=k+1}^n \frac{\partial f_i}{\partial y_i} \right) \frac{\partial f_k}{\partial w_k}$$

# Apport d'une librairie de réseaux de neurones



Bénéficier

- D'une implémentation optimisée de la backpropagation
- De nombreuses opérations implémentées
- De la prise en charge du GPU pour toutes ces opérations

Au delà des réseaux de neurones en eux mêmes :

- Optimiseurs (Adam, RMSProp, ...)
- Gestion des datasets
- Outils de visualisation (Tensorboard, ...)



## **6 Fin de la partie théorique**

# Sources



- <https://github.com/karpathy/micrograd> Micrograd de Andrej Karpathy
- <https://dlvu.github.io/> Vrije Universiteit Amsterdam - Deep Learning course
- <https://mathworld.wolfram.com/DirectionalDerivative.html> Directional derivative